# Lab 6: Conditional Branching and Control Flow in MIPS

# Learning Objectives

- Explain how branch instructions control program flow in MIPS.
- Use `beq` and `bne` to implement conditional decisions.
- Use `slt` and `slti` to compare values.
- Translate C-style `if` and `if–else` statements into MIPS.
- Implement while loops using branch and jump instructions.
- Trace program execution using QtSPIM.

# Common Arithmetic and Comparison Instructions

| Instruction | Meaning |
| --- | --- |
| `add rd, rs, rt` | Add two registers |
| `sub rd, rs, rt` | Subtract two registers |
| `mul rd, rs, rt` | Multiply register values |
| `sll rd, rt, shamt` | Shift left |
| `srl rd, rt, shamt` | Shift right logical |
| `sra rd, rt, shamt` | Shift right arithmetic |
| `slt rd, rs, rt` | Set `rd = 1` if `rs < rt`; otherwise 0 |
| `slti rt, rs, imm` | Set `rt = 1` if `rs < immediate`; otherwise 0 |

- Remember: slt and slti produce a value of either 0 or 1.

# Memory Access and System Calls

- Memory Access Instructions

| Instruction | Meaning |
| --- | --- |
| `lw rt, label` | Load a word from memory |
| `sw rt, label` | Store a word in memory |

- Common QtSPIM System Calls

| $v0 | Service |
| --- | --- |
| 1 | Print integer |
| 4 | Print string |
| 10 | Exit program |

- Example: Print an Integer

```
move $a0, $s2
li   $v0, 1
syscall
```

# Common Branch Instructions

| Instruction | Meaning | Condition |
|---|---|---|
| `beq rs, rt, label` | Branch if equal | `rs == rt` |
| `bne rs, rt, label` | Branch if not equal | `rs != rt` |

Example 1: Branch if Equal

```
    li  $t0, 5
    li  $t1, 5

    beq $t0, $t1, L1
    addi $s0, $s0, 1    # Skipped

    L1:
        # Branch target
```

Example 2: Branch if Not Equal

```
    li  $t0, 5
    li  $t1, 7

    bne $t0, $t1, L1
    addi $s0, $s0, 1    # Skipped

    L1:
        # Branch target
```

- beq and bne compare registers for equality or inequality. For conditions such as less than, we can use slt.

# Common MIPS Registers

| Register | Typical Use |
|---|---|
| `$t0–$t9` | Temporary values |
| `$s0–$s7` | Saved registers |
| `$zero` | Constant value `0` |
| `$a0–$a3` | Function arguments |
| `$v0` | Return value or syscall service |

- **Example:**

```
li  $s0, 5
li  $s1, 3

slt $t0, $s0, $s1
```

- Since `5 < 3` is `false`:

```
$t0 = 0
```

- If the comparison were true, `$t0` would contain `1`.

# Conditional Operations in MIPS

Programs use conditional operations to make decisions and repeat instructions.

- C-style If–Else

```
if (a == b)
    x = x + 1;
else
    x = x - 1;
```

- C-style While Loop

```
while (i < 5)
    i++;
```

**MIPS Control-Flow Instructions**

| Instruction | Purpose |
|---|---|
| `beq` | Branch when two registers are equal |
| `bne` | Branch when two registers are different |
| `slt` | Test whether one register is less than another |
| `j` | Jump unconditionally |

- Important: A label identifies a location in the program that a branch or jump can target.

# Example 1: `If-Else If-Else`

- Translate the following C-style program into MIPS assembly.

```
int a = 5;
int b = 3;
int result;

if (a < b)
{
    result = a + b;
}
else if (a == b)
{
    result = a * b;
}
else
{
    result = a - b;
}
```

Register Mapping

- `a` → `$s0`
- `b` → `$s1`
- `result` → `$s2`

- Requirements: Use `slt`, `beq` or `bne`, `labels`, and `j`.
- Expected Result: Since `5 < 3` is false and `5 == 3` is false, the final result is `5 - 3 = 2`.

# MIPS Solution

```
.data
a:       .word 5
b:       .word 3
result:  .word 0
newline: .asciiz "\n"

.text
.globl main

main:
    # Load a and b
    lw $s0, a
    lw $s1, b

    # if (a < b)
    slt $t0, $s0, $s1
    bne $t0, $zero, LESS

    # else if (a == b)
    beq $s0, $s1, EQUAL

    # else: result = a - b
    sub $s2, $s0, $s1
    j STORE

LESS:
    add $s2, $s0, $s1
    j STORE

EQUAL:
    mul $s2, $s0, $s1

STORE:
    # Store result
    sw $s2, result

    # Print result
    move $a0, $s2
    li   $v0, 1
    syscall

    # Print newline
    la $a0, newline
    li $v0, 4
    syscall

    # Exit
    li $v0, 10
    syscall
```

**Output:** 2

# Example 2: `While Loop` with `If-Else`

- Translate the following C-style program into MIPS assembly.

```
int i = 0;
int sum = 0;

while (i < 5)
{
    if (i % 2 == 0)
        sum = sum + i;
    else
        sum = sum - i;

    i++;
}
```

```
i = 0 → sum =  0
i = 1 → sum = -1
i = 2 → sum =  1
i = 3 → sum = -2
i = 4 → sum =  2
```
**Result: 2**

Program Logic

- If i is even, add `i` to `sum`.
- If i is odd, subtract `i` from `sum`.
- Continue `while i < 5`.

Register Mapping

`i → $t0`

`sum → $t1`

# MIPS Solution

```
.data
newline: .asciiz "\n"

.text
.globl main
main:
    li $t0, 0       # i = 0
    li $t1, 0       # sum = 0
    li $t2, 5       # loop limit

LOOP:
    # while (i < 5)
    slt $t3, $t0, $t2
    beq $t3, $zero, END

    # Check whether i is even
    andi $t4, $t0, 1
    beq  $t4, $zero, EVEN
ODD:
    sub $t1, $t1, $t0
    j CONTINUE

EVEN:
    add $t1, $t1, $t0

CONTINUE:
    addi $t0, $t0, 1
    j LOOP

END:
    # Print sum
    move $a0, $t1
    li   $v0, 1
    syscall

    # Print newline
    la $a0, newline
    li $v0, 4
    syscall

    # Exit
    li $v0, 10
    syscall
```

**Output**

2

# In-Class Task 1: `If–Else`

- Translate the following C-style program into MIPS assembly.

```
int x = 8;
int y = 5;
int z;

if (x < y)
    z = x + y;
else
    z = x - y;
```

- Register Mapping
  - `x → $s0`
  - `y → $s1`
  - `z → $s2`
- Requirements
  1. Load x and y.
  2. Compare the values using slt.
  3. Use a branch instruction to implement if–else.
  4. Store the final result in z.
  5. Print z in QtSPIM.

Expected Output

Since 8 < 5 is false, the else block executes:

`z = 8 – 5 = 3`

# In-Class Task 2: While Loop and If–Else

- Translate the following C-style program into MIPS assembly.

```
int i = 0;
int sum = 0;
while (i < 6)
{
    if (i < 3)
        sum = sum + i;
    else
        sum = sum - i;
    i++;
}
```

Register Mapping

```
i → $t0
sum → $t1
```

- **Requirements**
  1. Implement the `while (i < 6)` condition.
  2. Implement the `if (i < 3)` condition.
  3. Update `sum` correctly.
  4. Increment `i` after each iteration.
  5. Use `slt`, `beq` or `bne`, and `labels`.
  6. Print the final value of `sum`.

Expected Calculation

```
i = 0 → sum =  0
i = 1 → sum =  1
i = 2 → sum =  3
i = 3 → sum =  0
i = 4 → sum = -4
i = 5 → sum = -9
```

Result: −9

# Summary: Conditional Branching in MIPS

`beq:` Branches when two registers contain equal values.

`bne:` Branches when two registers contain different values.

`slt:` Sets a register to 1 if one value is less than another; otherwise 0.

`slti:` Compares a register against an immediate value.

`j:` Transfers control unconditionally to a label.

`Labels:` Identify locations that branches and jumps can target.

MIPS combines comparison instructions, conditional branches, unconditional jumps, and labels to implement if, if–else, and while constructs.